

NÚCLEO DE PESQUISA: DÍVIDA TÉCNICA E DÍVIDA COGNITIVA NO DESENVOLVIMENTO DE SOFTWARE ASSISTIDO POR IA GENERATIVA

Esta pesquisa acadêmica investiga os impactos de longo prazo do uso intensivo de IA Generativa na manutenibilidade do software, focando na interseção estrutural entre a Dívida Técnica (código) e a Dívida Cognitiva (compreensão humana).

A. DELIMITAÇÃO

O estudo explora como a automação da escrita de código afeta a evolução futura do software. A pesquisa não assume que 'código de IA é inerentemente ruim', mas investiga empiricamente as circunstâncias sob as quais a geração assistida eleva a Dívida Técnica (acoplamento, duplicação, complexidade ciclomática) e instaura a Dívida Cognitiva (perda do modelo mental do sistema e alienação arquitetural por parte dos desenvolvedores).

B. PERGUNTAS DE PESQUISA

1. Como a Dívida Técnica (estrutural) e a Dívida Cognitiva (mental) se manifestam e se diferenciam em bases de código geradas majoritariamente por IA?
2. Quais evidências empíricas indicam que o uso de LLMs afeta a duplicação, a legibilidade e a manutenibilidade do software a longo prazo?
3. Quais mecanismos metodológicos de Engenharia de Software (testes, documentação, rastreabilidade) são necessários para mitigar essas dívidas em projetos assistidos por IA?

C. SUBTEMAS INVESTIGADOS

- Conceitos Fundamentais: Dívida técnica intencional vs. não intencional, causas e consequências da dívida.
- Qualidade de Código e IA: Legibilidade, complexidade, duplicação (DRY), acoplamento e arquitetura fragmentada gerada por LLMs.
- Dívida Cognitiva: Alienação do código, dependência do gerador, perda de conhecimento sistêmico e dificuldade de compreensão.

- Efeitos a Longo Prazo: Impacto na manutenção, evolução do sistema e onboarding de novos desenvolvedores.

D. ESTRATÉGIA E CRITÉRIOS DE SELEÇÃO

Foram priorizados estudos de 2021 a 2026, com ênfase em métricas quantitativas de manutenção de código (code churn, duplicação) em ambientes utilizando GitHub Copilot e ChatGPT. Fontes clássicas (1992 e 2009) foram excepcionalmente integradas pois estabelecem a taxonomia financeira insubstituível da 'Dívida Técnica', sem a qual a análise moderna perde seu referencial teórico e econômico.

E. FONTES E F. EVIDÊNCIAS EXTRAÍDAS

Fonte 1 (Clássica Fundacional)

Autor(es): CUNNINGHAM, W. / FOWLER, M.

Ano: 1992 / 2009

Título: The WyCash Portfolio Management System (Cunningham) / Technical Debt Quadrant (Fowler)

Tipo: Artigo Seminal e Postulado de Engenharia.

Justificativa de Inclusão: Fundamental para definir o conceito. Cunningham introduziu a metáfora financeira (escrever código rápido gera 'juros' de manutenção futura), e Fowler categorizou a dívida em intencional/não intencional e prudente/imprudente. Não é possível debater dívida moderna sem esta taxonomia teórica.

Evidências: A dívida técnica imprudente e não intencional é a mais destrutiva, ocorrendo quando a equipe não compreende a arquitetura e acopla componentes incorretamente.

Fonte 2

Autor(es): GITCLEAR (MACKEY, A. et al.)

Ano: 2024

Título: Coding on Copilot: 2024 Data Suggests Downward Pressure on Code Quality

Periódico: GitClear Research Report (Whitepaper Técnico baseado em dados empíricos de repositórios reais)

Tipo: Estudo Empírico / Análise de Repositórios.

Evidências: Avaliou mais de 150 milhões de linhas de código pós-Copilot (2023-2024). Provou empiricamente que o 'code churn' (código escrito e logo descartado/reescrito) aumentou drasticamente, assim como a duplicação de código ('copy-pasted code'). A IA induz os desenvolvedores a repetirem blocos em vez de refatorarem em funções reutilizáveis (violação do princípio DRY), elevando a dívida técnica passiva.

Fonte 3

Autor(es): OUH, E. L. et al.

Ano: 2023

Título: ChatGPT, AI-Generated Code, and Technical Debt: An Empirical Study

Periódico: Anais da IEEE/ACM

Tipo: Estudo Empírico.

Evidências: Demonstra que a IA não gera inerentemente 'código ruim', mas gera 'código localmente ótimo e globalmente subótimo'. Ou seja, a função isolada funciona bem, mas frequentemente falha em se integrar aos padrões arquiteturais do repositório, criando uma arquitetura fragmentada e elevando o acoplamento.

Fonte 4

Autor(es): BRANCO, B. M. C. et al.

Ano: 2025

Título: Avaliando o Papel do GPT-4o na Aprendizagem de Engenharia de Software: Um Relato de Experiência

Periódico: Anais do SBES

Tipo: Estudo Empírico e Relato Qualitativo.

Evidências: Cunhou empiricamente a transição para a 'Dívida Cognitiva'. Estudantes e desenvolvedores que usam IA intensivamente delegam o raciocínio estrutural. O código funciona, mas a equipe perde a 'compreensão do porquê', gerando uma dependência estrita do gerador para futuras manutenções.

G. ANÁLISE CRÍTICA

A Dívida Técnica em código de IA não decorre de 'sintaxe errada', mas sim de design pobre em nível macro. A análise dos dados empíricos de 2024 desmistifica a ideia

de que a IA escreve código estruturalmente brilhante: ela é uma 'copiadora sofisticada' que favorece adições (additions) sobre refatorações (refactors). O maior risco comprovado é a duplicação de código. Além disso, a literatura recente solidifica o conceito de 'Dívida Cognitiva'. Enquanto a dívida técnica reside nos arquivos do projeto, a dívida cognitiva reside no cérebro da equipe: é o custo mental de ter que decifrar um sistema 'caixa preta' escrito por uma máquina, caso ocorra um bug e o LLM não consiga consertar o contexto.

H. COMPOSIÇÃO ACADÊMICA ÚNICA

A Dívida Técnica, definida originalmente por Ward Cunningham em 1992, é uma metáfora financeira que ilustra o custo futuro incorrido quando equipes de software optam por soluções de codificação rápidas e subótimas em vez de abordagens arquiteturalmente corretas. Fowler complementou essa visão destacando que a dívida pode ser contraída de forma intencional (para ganhar mercado rapidamente) ou não intencional (por ignorância técnica). Em paralelo, emerge o conceito de Dívida Cognitiva, definida como a lacuna entre a complexidade sistêmica de um software em produção e a capacidade mental da equipe de desenvolvimento de compreendê-lo e iterar sobre ele com segurança (BRANCO et al., 2025).

A relação entre essas duas dívidas é umbilicalmente retroalimentada, e a Inteligência Artificial Generativa atua como um catalisador hipertrófico de ambas. Dados empíricos recentes sobre bases de código assistidas por IA (como os levantamentos de 2024 do GitClear) não confirmam a hipótese simplista de que 'IA gera código quebrado'. Pelo contrário, a IA gera funções que compilam e passam em testes triviais. O problema surge na macroarquitetura: a GenAI exibe uma tendência de gerar Dívida Técnica Não Intencional ao favorecer a duplicação excessiva de blocos de código em detrimento da abstração em funções reutilizáveis (DRY). A arquitetura torna-se fragmentada porque os LLMs otimizam a resposta para o prompt imediato, ignorando o design sistêmico global do repositório (OUH et al., 2023).

Consequentemente, essa proliferação rápida de código sintaticamente complexo — que não foi genuinamente pensado pela equipe humana — instaura a Dívida

Cognitiva. Os desenvolvedores transformam-se em operadores de ferramentas, aprovando Pull Requests volumosos sem profunda compreensão. Se um bug sutil de estado concorrente ocorrer em produção um ano depois, o 'conhecimento perdido' exigirá um esforço colossal de engenharia reversa. O sistema torna-se dependente do gerador algorítmico original.

Contudo, a IA não é uma produtora inevitável de dívidas. A literatura sustenta que, sob governança estrita, a IA pode atuar como um amortecedor de dívida técnica, auxiliando na geração maciça de documentação (explicabilidade), cobertura de testes automatizados e identificação de rotinas depreciadas. Para reduzir os riscos, a metodologia deve proibir que a velocidade da geração de código exceda a velocidade da auditoria humana.

I. IMPLICAÇÕES PARA A TG

No desenvolvimento do sistema acadêmico da FATEC, os conceitos de Dívidas Técnica e Cognitiva exigem mecanismos concretos na metodologia do grupo:

1. Combate à Duplicação (Dívida Técnica): O uso do Gemini/Copilot não pode justificar a duplicação de lógicas de consulta (ex: regras de restrição de salas). Se a IA sugerir copiar e colar um bloco estrutural, o grupo deve intervir e refatorar a lógica em um Serviço Único do Spring Boot (@Service).

2. Auditoria e Retenção do Conhecimento (Dívida Cognitiva): O grupo deverá implementar 'Sessões de Compartilhamento' ou 'Pair Programming'. Quem utilizou a IA para gerar um módulo deve ser capaz de explicar a arquitetura desse módulo sem a ajuda do código. Isso documenta que o modelo mental do sistema ainda pertence aos humanos da FATEC, não à máquina.

3. Testes como Rastreabilidade: Toda funcionalidade que for gerada rapidamente por IA deve ser obrigatoriamente coberta por testes automatizados estruturados. Os testes servem como o 'pagamento contínuo dos juros' da Dívida Técnica.

J. REFERÊNCIAS ABNT COMPLETAS

BRANCO, B. M. C.; MELO NETO, A. L. de; AVELINO, G. A.; MACHADO, V. P. Avaliando o Papel do GPT-4o na Aprendizagem de Engenharia de Software: Um Relato de Experiência. Em: Anais do Simpósio Brasileiro de Engenharia de Software (SBES). Recife: SBC, 2025.

CUNNINGHAM, W. The WyCash Portfolio Management System. ACM SIGPLAN OOPS Messenger, Nova York, v. 4, n. 2, p. 29-30, dez. 1992. (Fonte Clássica Fundacional).

FOWLER, M. Technical Debt Quadrant. MartinFowler.com, [S. l.], out. 2009. Disponível em: <https://martinfowler.com/bliki/TechnicalDebtQuadrant.html>. Acesso em: 6 out. 2026. (Fonte Clássica Fundacional).

GITCLEAR. Coding on Copilot: 2024 Data Suggests Downward Pressure on Code Quality. Seattle: GitClear Research, jan. 2024. Relatório Técnico.

OUH, E. L. et al. ChatGPT, AI-Generated Code, and Technical Debt: An Empirical Study. Em: Anais da IEEE/ACM. Nova York: IEEE, 2023.

K. LACUNAS DE INVESTIGAÇÃO

- A maioria das ferramentas automatizadas de medição de qualidade de código (como SonarQube) foi calibrada para métricas da era pré-GenAI; existe uma lacuna acadêmica sobre como construir métricas estáticas que identifiquem a 'assinatura arquitetural fragmentada' típica de LLMs.
- Faltam estudos longitudinais focados estritamente na curva de degradação da Dívida Cognitiva: mensurar exatamente em quantos meses o time humano se torna completamente incapaz de manter o sistema sem o LLM.